

COMM116

Generative AI Applications

Section 3: World Models and Autonomous Systems

Dr Xiaoyang Wang

Department of Computer Science

x.wang7@exeter.ac.uk

Outline



RL, MBRL, Model predictive control (MPC)

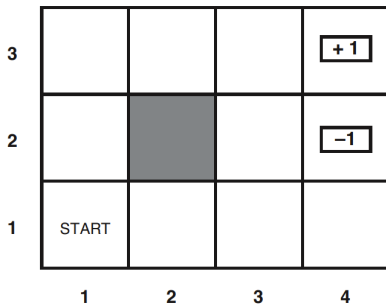
World models, Dreamer

Application - Autonomous Vehicles

RL Recap

- Formulate decision-making problems as Markov Decision Processes (MDPs).
- Understand the RL objective, value functions, and policy optimization intuition.
- Explain what makes RL *model-based* and why MBRL improve data efficiency.
- Understand MPC as receding-horizon planning with constraints.

Example: Grid World

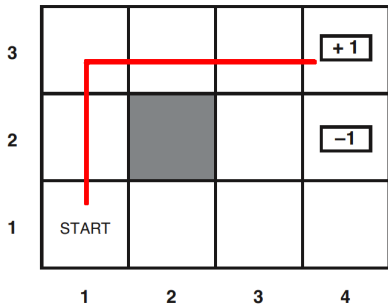


Start: (1,1)

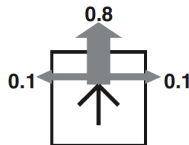
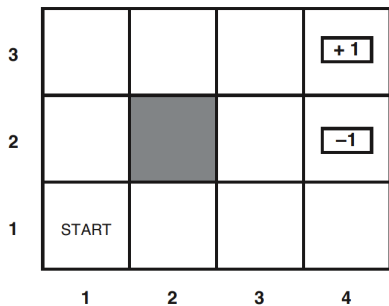
Goal: (4, 3) and (4, 2)

If the robot bumps the wall, it stays in the same square.

Example: Grid World



Example: Stochastic Grid World



The probability of reaching (4,3) following the previous solution:

$$0.8^5 = 0.32768$$

RL setup: Markov Decision Process (MDP)

MDP

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$$

- $\mathcal{S}$: state space, $\mathcal{A}$: action space
- $P(\mathbf{s}'|\mathbf{s}, \mathbf{a})$: transition dynamics
- $r(\mathbf{s}, \mathbf{a})$: reward signal
- $\gamma \in [0, 1]$: discount factor
- Policy $\pi(\mathbf{a}|\mathbf{s})$ chooses actions.
- Trajectory: $\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \mathbf{a}_1, \dots)$
- Goal: maximize expected discounted return.

RL objective

Expected return

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right]$$

- **Model-free RL**: optimize policy/value without explicitly using P for planning.
- **Model-based RL**: learn or use $\hat{P}$ to predict futures and improve decisions.

Design question

Do we spend compute on *real-world data collection*, or on *simulation/planning* using a model?

Value functions

Definitions

$$V^{\pi}(\mathbf{s}) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid \mathbf{s}_0 = \mathbf{s} \right]$$

$$Q^{\pi}(\mathbf{s}, \mathbf{a}) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid \mathbf{s}_0 = \mathbf{s}, \mathbf{a}_0 = \mathbf{a} \right]$$

Bellman equations

$$V^{\pi}(\mathbf{s}) = \mathbb{E}_{\mathbf{a} \sim \pi, \mathbf{s}' \sim P} [r(\mathbf{s}, \mathbf{a}) + \gamma V^{\pi}(\mathbf{s}')]]$$

$$Q^{\pi}(\mathbf{s}, \mathbf{a}) = \mathbb{E}_{\mathbf{s}' \sim P} [r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi} Q^{\pi}(\mathbf{s}', \mathbf{a}')]]$$

RL algorithm families

Value-based

Learn $Q(\mathbf{s}, \mathbf{a})$ and act greedily:

$$\mathbf{a}_t \approx \arg \max_{\mathbf{a}} Q(\mathbf{s}_t, \mathbf{a})$$

Examples:

- Q-learning
- DQN
- Double DQN

Policy-based, Actor-Critic

Directly optimize $\pi_{\theta}(\mathbf{a}|\mathbf{s})$:

$$\nabla_{\theta} J(\pi_{\theta}) \propto \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(\mathbf{a}|\mathbf{s}) \hat{A}(\mathbf{s}, \mathbf{a})]$$

Examples:

- REINFORCE
- PPO
- SAC / TD3 (continuous control)

Model-free methods.

Model-based vs Model-free

Model-free

- Low sample efficiency
- Directly working on the value function / policy
- In practice, easier to implement and tune

Model-based

- Having a model allows the agent to *plan*
- The real model of the environment is usually not available to the agent, also difficult to learn
- Model bias
- High sample efficiency



Model-based RL

- **Data efficiency:** reuse data through imagined rollouts.
- **Safety (potentially):** reject unsafe plans before executing.
- **Adaptation:** replan online when conditions change.
- **Interpretability:** inspect predicted trajectories and costs.

Model: transition dynamics + reward

$$\hat{\mathbf{s}}_{t+1} = \hat{f}_{\psi}(\mathbf{s}_t, \mathbf{a}_t), \quad \hat{r}_t = \hat{r}_{\psi}(\mathbf{s}_t, \mathbf{a}_t)$$

or probabilistic versions

$$\hat{P}_{\psi}(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

Learning the model

Given dataset $\mathcal{D} = \{(\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{t+1}, r_t)\}$:

Deterministic model fitting

$$\min_{\psi} \sum_{(\mathbf{s}, \mathbf{a}, \mathbf{s}') \in \mathcal{D}} \|\mathbf{s}' - \hat{f}_{\psi}(\mathbf{s}, \mathbf{a})\|_2^2$$

Probabilistic model fitting (maximum likelihood)

$$\min_{\psi} - \sum_{(\mathbf{s}, \mathbf{a}, \mathbf{s}') \in \mathcal{D}} \log \hat{P}_{\psi}(\mathbf{s}' | \mathbf{s}, \mathbf{a})$$

- Often also learn a reward model $\hat{r}_{\psi}$.
- In image-based tasks, sometimes learn latent state $\mathbf{x}_t$ and latent dynamics.

Generic MBRL loop

- ① Collect transitions in the real environment.
- ② Fit/update world model $\hat{f}_{\psi}$ (and possibly uncertainty estimate).
- ③ Plan with the model (or train a policy using model rollouts).
- ④ Execute selected action(s) in the real world.
- ⑤ Add new data and repeat.

Common planning styles

- **Shooting / sampling**: sample action sequences, roll out in model, keep best.
- **Gradient-based planning**: optimize actions via backprop through the model.
- **MPC**: finite-horizon optimization, apply first action, then replan.



Model error and compounding drift

- One-step prediction can be accurate, but long rollouts can drift.
- Planner may exploit model inaccuracies (*model bias*).
- Distribution shift: model trained on seen states, planner visits unseen states.

Typical mitigations

- Short planning horizons (MPC)
- Ensembles / uncertainty penalties
- Frequent replanning



Model Predictive Control (MPC): intuition

At each time step:

- ① Predict future states over horizon H using a model.
 - ② Optimize an action sequence for a finite-horizon objective.
 - ③ Execute only the first action.
 - ④ Observe new state and repeat.
- “Predictive” because it uses a model to forecast trajectories.
 - “Control” because it closes the loop by replanning after each observation.



MPC optimization problem

Finite-horizon optimal control

$$\max_{a_{t:t+\tau}} \mathbb{E}_{s_{t'+1} \sim p(s_{t'+1} | s_{t'}, a_{t'})} \left[\sum_{t'=t}^{t+\tau} r(s_{t'}, a_{t'}) \right]$$

subject to additional constraints.

MPC in autonomous systems

- **Constraints:** natural way to encode constraints and limits.
- **Reactivity:** replan as the environment changes.
- **Works with models of varying fidelity:** linear, nonlinear, learned.

Examples

- Autonomous driving: lane keeping, obstacle avoidance, speed planning
- Drones: trajectory tracking under constraints

RL, MBRL, MPC

Aspect	Model-free RL	MBRL	MPC
Model of dynamics	Not explicit	Learned (or known)	Required (known or learned)
Decision style	Learned policy/value	Planning and/or learned policy	Online optimization
Data efficiency	Often low	Often higher	N/A (control method; depends on model source)
Real-time compute	Low (policy inference)	Medium-high	Medium-high
Typical strength	Generality	Sample efficiency + planning	Apply constraints

Early MBRL and world-model ideas

- **Sutton (1990), Dyna**: integrates learning, planning, and acting using a learned model.
- **Schmidhuber (1990, 2015, 2018)**: controller + recurrent world model for “plan-ahead” behavior.
- **PILCO (2011)**: probabilistic dynamics model with uncertainty-aware long-term planning.
- **I2A (2017)**: imagination-augmented agents use learned rollouts to improve decisions.

Modern “world models” did not appear from nowhere; they build on earlier MBRL and predictive-model ideas in control, RL, and cognitive science.

World models

World models are *predictive simulators* used for:

- **Control** (MPC)
 - **Planning** (search over futures)
 - **Learning** (synthetic data, imagination, representation learning)
-
- Classical control: model is often hand-designed.
 - MBRL: model is learned from data.
 - Modern world models: model may be latent, high-dimensional, and jointly learned with policy/value.

References

- Sutton, R.S., 1991. Dyna, an integrated architecture for learning, planning, and reacting. ACM Sigart Bulletin, 2(4), pp.160-163.
- Deisenroth, M. and Rasmussen, C.E., 2011. PILCO: A model-based and data-efficient approach to policy search. In Proceedings of the 28th International Conference on machine learning (ICML-11) (pp. 465-472).
- Racanière, S., Weber, T., Reichert, D., Buesing, L., Guez, A., Jimenez Rezende, D., Puigdomènech Badia, A., Vinyals, O., Heess, N., Li, Y. and Pascanu, R., 2017. Imagination-augmented agents for deep reinforcement learning. Advances in neural information processing systems, 30.
- Chua, K., Calandra, R., McAllister, R. and Levine, S., 2018. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. Advances in neural information processing systems, 31.

World models, Dreamer

Background

World models: Ha, D. and Schmidhuber, J., 2018. "Recurrent world models facilitate policy evolution". Advances in neural information processing systems, 31.

"Humans develop a mental model of the world based on what they are able to perceive, learning abstract representations of both spatial and temporal aspects of sensory inputs."

- Emphasize "perception" and "representation".
- Internal model - within the **agent**; incorporates memory.
- Temporal information.

Background

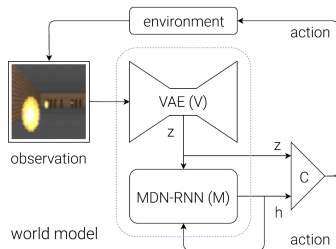
World models: Ha, D. and Schmidhuber, J., 2018. "Recurrent world models facilitate policy evolution". Advances in neural information processing systems, 31.

Core ideas:

- **Modular Agent Architecture (V-M-C).**
 - A visual sensory component (V) compresses observation into a representation. VAE.
 - A predictive model (recurrent) for memory (M).
 - Controller (C) for decision making, given vision and memory.
- The world model - M, or **V+M**. V captures the current observation, while M captures the temporal information (predictive model).
- Decoupled training of V, M and C. C is light-weighted.
- Showed that agents can learn inside of their own dreams - learning with their own generated environment.

Background

World models



```
def rollout(controller):  
    ''' env, rnn, vae are '''  
    ''' global variables '''  
    obs = env.reset()  
    h = rnn.initial_state()  
    done = False  
    cumulative_reward = 0  
    while not done:  
        z = vae.encode(obs)  
        a = controller.action([z, h])  
        obs, reward, done = env.step(a)  
        cumulative_reward += reward  
        h = rnn.forward([a, z, h])  
    return cumulative_reward
```

Left: Agent model, how V, M, and C interacts with the environment. Right: Pseudocode for how the agent model works with a Gym env. Figure from Ha, D. & Schmidhuber, J. (2018).

Ha, D. and Schmidhuber, J., 2018. "Recurrent world models facilitate policy evolution". Advances in neural information processing systems, 31.

Background

World models

- ① Version 1 - Learning with a pretrained world model. V and M are trained with rollouts from a random policy (offline data). Only the controller C has access to the reward.

Ha, D. and Schmidhuber, J., 2018. "Recurrent world models facilitate policy evolution". Advances in neural information processing systems, 31.

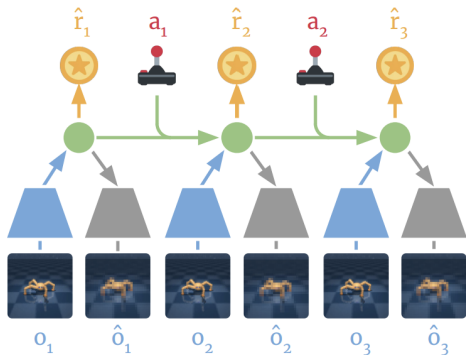
Background

World models

- ① Version 1 - Learning with a pretrained world model. V and M are trained with rollouts from a random policy (offline data). Only the controller C has access to the reward.
- ② Version 2 - **Train the agent in the world model** (without access to the real environment).
 - M needs to predict not only the next frame, but also the reward (and episode termination, in most cases).
 - Lose V - simulation is entirely in the latent space, assumes that V is trained well, and M predicts latent of V very well.
 - The controller has access to hidden states (memory), as the world model is *inside the agent*.
 - The controller C might exploit the world model; need to adjust the uncertainty of M .

Ha, D. and Schmidhuber, J., 2018. "Recurrent world models facilitate policy evolution". Advances in neural information processing systems, 31.

Latent Dynamics



Recurrent model: $h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1})$

Representation model: $z_t \sim q_\phi(\cdot | h_t, o_t)$

Transition model: $\hat{z}_t \sim p_\phi(\cdot | h_t)$

Reward model: $\hat{r}_t \sim p_\phi(\cdot | h_t, z_t)$

Figure from Dreamer (Hafner et al., 2019)

q for distributions that generate samples with the real observation; p for their approximations that enable latent imagination. f is the RNN's recurrent transition, i.e., deterministic.

Hafner, D., Lillicrap, T., Ba, J. and Norouzi, M., 2019. Dream to control: Learning behaviors by latent imagination. arXiv preprint arXiv:1912.01603.

Latent Dynamics

Transition model

$$\hat{z}_t \sim p_\phi(\cdot \mid h_t) = p_\phi(\cdot \mid h_{t-1}, z_{t-1}, a_{t-1})$$

A **generative model**, i.e., allows the model to sample from a distribution over possible future latent states.

The generative model works in a **compact latent space**, i.e., learnt by a VAE over high dimensional observations (e.g., acquired from image sequences).

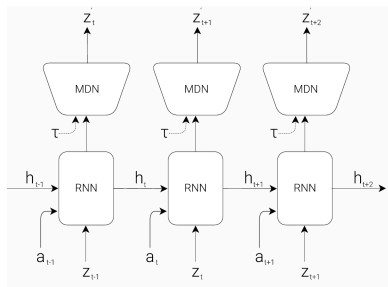
Latent Dynamics

Transition model

$$\hat{z}_t \sim p_\phi(\cdot | h_t) = p_\phi(\cdot | h_{t-1}, z_{t-1}, a_{t-1})$$

A **generative model**, i.e., allows the model to sample from a distribution over possible future latent states.

The generative model works in a **compact latent space**, i.e., learnt by a VAE over high dimensional observations (e.g., acquired from image sequences).



An RNN with a mixture density network (MDN). Figure from Ha, D. & Schmidhuber, J. (2018). τ is the temperature parameter controlling the stochasticity.

Imagination and Planning

With the latent dynamics, the agent “imagines” trajectories. Assuming τ is the time index, imagination start at the true states z_t , then follow the transition model

$$z_\tau \sim p(z_\tau \mid z_{\tau-1}, h_{\tau-1}, a_{\tau-1}),$$

reward model

$$r_\tau \sim p(r_\tau \mid z_\tau, h_\tau),$$

and a policy

$$a_\tau \sim p(a_\tau \mid z_\tau, h_\tau).$$

The objective is to maximize expected imagined rewards with respect to the policy.

$$\mathbb{E}_q \left(\sum_{\tau=t}^{\infty} \gamma^{\tau-t} r_\tau \right)$$

The **world model** is an imagination engine that facilitates mental simulation.

Dreamer V3 (Hafner et al., 2023)

- Dreamer learns a compact latent dynamics model and performs policy optimization entirely in latent space.
- Dreamer relies on a recurrent state-space model (RSSM) that factorizes the environment into deterministic recurrent states and stochastic latent representations.
- Stochasticity in transition model - the stochastic latent state is defined by **discrete** categorical variables.

Hafner, D., Pasukonis, J., Ba, J. and Lillicrap, T., 2023. Mastering diverse domains through world models. arXiv preprint arXiv:2301.04104.



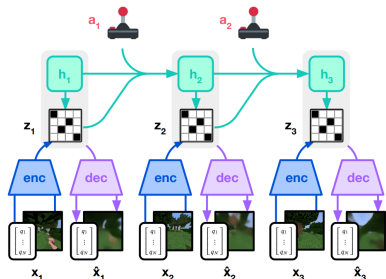
DreamerV3

- It combines:
 - a learned **world model** (RSSM),
 - an **actor** (policy),
 - a **critic** (value model).
- Key claim: strong performance across many tasks (Atari, ProcGen, DMLab, control, Minecraft) without per-domain retuning.

Core idea

Learn compact latent dynamics, imagine future trajectories in latent space, and improve behavior from imagined rollouts.

Dreamer



World model learning in Dreamer
(Hafner et al., 2023).

Recurrent model: $h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1})$

Transition model (prior): $\hat{z}_t \sim p_\phi(\cdot | h_t)$

Representation (posterior): $z_t \sim q_\phi(\cdot | h_t, x_t)$

Predictors: $\hat{x}_t, \hat{r}_t, \hat{\gamma}_t \sim p_\phi(\cdot | h_t, z_t)$

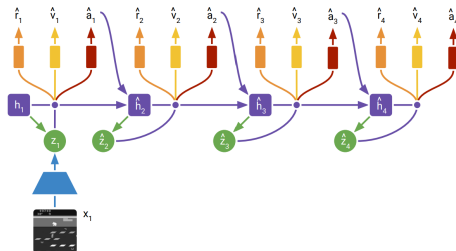
The KL divergence between the posterior and prior is minimized to train the model:

$$\mathcal{L}_{\text{KL}} = D_{\text{KL}}(q_\phi(z_t | h_t, x_t) \| p_\phi(\cdot | h_t)).$$

Hafner, D., Pasukonis, J., Ba, J. and Lillicrap, T., 2023. Mastering diverse domains through world models. arXiv preprint arXiv:2301.04104.

Dreamer

Behavior learning with the trained world model (Hafner et al., 2023)



- The transition model (prior) is used to roll forward in the latent space.
- Actor: Trained to output actions that maximize the value function prediction of the critic; Stochastic; With a policy entropy regularizer to encourage exploration.
- Critic: Trained to predict the value function; Deterministic.

Hafner, D., Pasukonis, J., Ba, J. and Lillicrap, T., 2023. Mastering diverse domains through world models. arXiv preprint arXiv:2301.04104.

RSSM (Recurrent State-Space Model)

Latent dynamics structure

The world model maintains a recurrent deterministic state h_t and stochastic latent z_t .

$$h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1}), \quad z_t \sim q_\phi(z_t | h_t, x_t)$$

$$\hat{z}_t \sim p_\phi(\hat{z}_t | h_t)$$

- Encoder maps observation x_t to latent z_t
- Decoder reconstructs x_t from (h_t, z_t)
- Auxiliary heads predict reward r_t and continuation c_t

Model “state” for control

$$s_t \triangleq (h_t, z_t)$$

Actor and critic operate on this latent state.

World model outputs and training losses

From (h_t, z_t) , Dreamer predicts:

- observation reconstruction $\hat{x}_t$
- reward $\hat{r}_t$
- continuation flag $\hat{c}_t \in \{0, 1\}$

Loss decomposition (high level)

$$\mathcal{L}_{\text{wm}} = \beta_{\text{pred}} \mathcal{L}_{\text{pred}} + \beta_{\text{dyn}} \mathcal{L}_{\text{dyn}} + \beta_{\text{rep}} \mathcal{L}_{\text{rep}}$$

- $\mathcal{L}_{\text{pred}}$: reconstruction + reward + continuation prediction
- $\mathcal{L}_{\text{dyn}}$: make prior dynamics predict posterior latents
- $\mathcal{L}_{\text{rep}}$: keep latents predictable (representation regularization)

Imagined rollouts in latent space

Key idea

Dreamer does not plan with online trajectory optimization at test time. Instead, it learns actor/critic from **imagined latent trajectories**.

- Start from replay states (h_t, z_t)
- Roll out latent dynamics with actor actions:

$$a_t \sim \pi_\theta(a_t \mid s_t), \quad s_{t+1} \sim p_\phi(\cdot \mid s_t, a_t)$$

- Predict rewards and continuation along imagined trajectory

Important distinction

Dreamer uses a **world model + actor-critic training**; it is not MPC. At deployment, actions come from the learned actor (fast policy inference).

Critic learning

Critic predicts a return distribution

The critic models a distribution over returns R_t conditioned on latent state:

$$v_{\psi}(R_t | s_t)$$

and uses its expectation as the value estimate.

Target: bootstrapped λ -returns

$$R_t^{\lambda} = r_t + \gamma c_t((1 - \lambda)v_t + \lambda R_{t+1}^{\lambda})$$

- Critic is trained by maximum likelihood on return targets
- DreamerV3 uses a **distributional critic** (categorical bins), not a single Gaussian scalar

Discussion

- ① Why might latent imagination be more sample-efficient than model-free RL?
- ② What failure modes arise if the world model is wrong?
- ③ Dreamer uses a learned actor at inference time; when would MPC be preferable?

Discussion

- ① Why might latent imagination be more sample-efficient than model-free RL?
Training a world model and generating many imagined rollouts in latent space; latent vs pixel space;
- ② What failure modes arise if the world model is wrong?
The policy/planner can exploit model errors (model bias); Multi-step predictions can drift (compounding error); uncertainty may be underestimated, and the model may be poor in out-of-distribution regions visited by the learned policy.
- ③ Dreamer uses a learned actor at inference time; when would MPC be preferable?
When a reliable model is available and runtime optimization is acceptable; need explicit constraints.

Reference

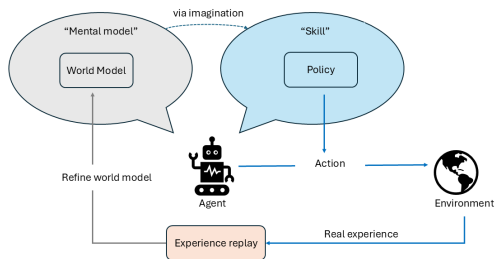
DreamerV3

- Hafner, Pasukonis, Ba, Lillicrap. *Mastering Diverse Domains through World Models* (DreamerV3), arXiv 2024.

Related Dreamers:

- Dreamer (2019), DreamerV2 (2020), DreamerV3 (2024)

Experience Collection



Experience $\rightarrow$ World Model $\rightarrow$ Imagination $\rightarrow$ Policy $\rightarrow$ Real Environment $\rightarrow$ Experience $\rightarrow \dots$

- Dreamer does not plan directly in the real environment.
- During interaction with the real environment, only the representation model is used to infer the latent state, which is then used by the actor.

Encourage Exploration

Motivations

- ① The predictive capacity of the world model is not used during interaction with the real environment.
- ② Consider environments with sparse reward (likely relates to some “key” and unknown states), can we steer the agent towards less explored states, to collect more informative experience data?
- ③ With a predictive mental model, it is possible to anticipate novel states rather than reacting retrospectively after a novel experience has occurred.



Exploring with High Entropy Plans

Methodology and Results

Research Question

World models (e.g., Dreamer series models), show that agents can learn with their own compact generative models (their imaginations/dreams). (Ha, D. & Schmidhuber, J., 2018)

Like explorers driven by imagination, can agents explore the real world more efficiently, with the guidance of their world model, to uncover new experiences that in return refine their inner world model?



Example: Existing work in autonomous driving

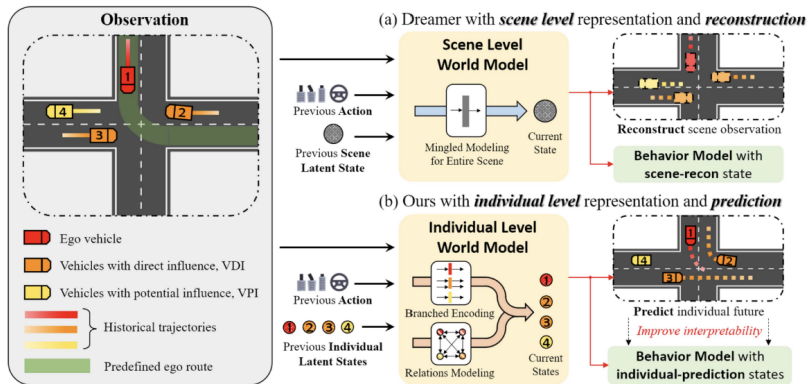
Predictive Individual World Model

Predictive individual world model (PIWM) (Gao et al., 2024)

- In interactive scenerios, such as autonomous vehicles (AVs) in urban scenerios, other road users' intentions are unknown (POMDP).
- World model approaches enable learning within imagination over a compact latent space. The representation is driven by scene-level reconstruction without modeling individual behavior in a scene.
- The transition model in world models do not naturally model interactions.
- Lacks long-term intention modeling; the predictive transition model might not be enough.

Gao, Y., Zhang, Q., Ding, D.W. and Zhao, D., 2024. Dream to Drive With Predictive Individual World Model. IEEE Transactions on Intelligent Vehicles.

Predictive Individual World Model



Gao, Y., Zhang, Q., Ding, D.W. and Zhao, D., 2024. Dream to Drive With Predictive Individual World Model. IEEE Transactions on Intelligent Vehicles.

Predictive Individual World Model

Predictive individual world model (PIWM) (Gao et al., 2024)

Q1: For interactive scenarios, with the transition model, how to encode interactions?

Individual-level transition modeling and **self-attention**.

Recall in Dreamer,

- Transition model (prior): $\hat{z}_t \sim p_\phi(\cdot \mid h_t)$.

With individual world models, the transition is

- $\hat{z}_t \sim p_\phi(\cdot \mid h_t, \text{self_attention})$.

Here the self_attention is calculated from the recurrent states (h_t) of all individuals.

Equivalent to maintaining individual “worlds” for interactive agents.

Gao, Y., Zhang, Q., Ding, D.W. and Zhao, D., 2024. Dream to Drive With Predictive Individual World Model. IEEE Transactions on Intelligent Vehicles.

Predictive Individual World Model

Predictive individual world model (PIWM) (Gao et al., 2024)

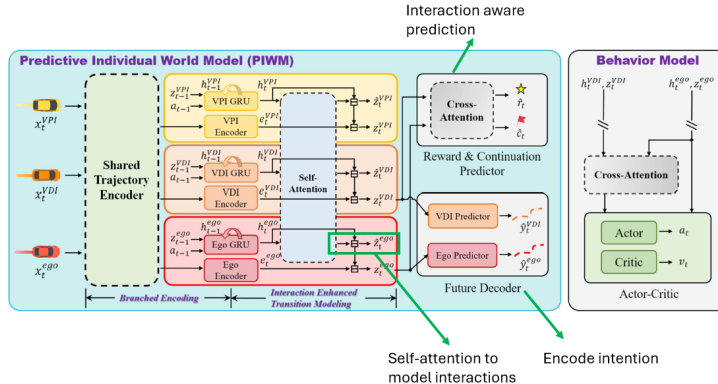
Q2: The latent state in world models contains memory of previous states and actions (*the history*). Could it also model intention (*the future*)?

Future decoding: use decoder to predict future trajectories over a 2-second horizon, forcing the latent state to implicitly learn about the intention.

Intention of surrounding agents are implicitly fed into the latent state via self-attention.

Predictive Individual World Model

Predictive individual world model (PIWM) (Gao et al., 2024)



Interaction-aware individual world model. Figure adapted from Gao et al., 2024.

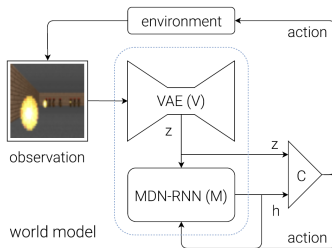
Challenges

- Model intentions and preferences of interactive agents. Latent representation for human behaviour?
- Beyond autonomous driving - collaborative human-robot interaction, and other tasks?



Figure from Habitat 3, <https://aihabitat.org/habitat3/>.

Challenges



Ha, D. & Schmidhuber, J. (2018).

- Is it the best way to design an agent with a mental model and memory, with the ability to learn and interact with the external world?

Summary



Summary

- MBRL.
- World models, learning with imagination.
- Applications: autonomous driving, autonomous robots and beyond.

References

- Ha, D. and Schmidhuber, J., 2018. Recurrent world models facilitate policy evolution. Advances in neural information processing systems, 31.
- Luo, F.M., Xu, T., Lai, H., Chen, X.H., Zhang, W. and Yu, Y., 2024. A survey on model-based reinforcement learning. Science China Information Sciences, 67(2), p.121101.
- Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H. and Davidson, J., 2019, May. Learning latent dynamics for planning from pixels. In International conference on machine learning (pp. 2555-2565). PMLR.
- Hafner, D., Pasukonis, J., Ba, J. and Lillicrap, T., 2023. Mastering diverse domains through world models. arXiv preprint arXiv:2301.04104.
- Sundar, A., Luo, C. and Wang, X., 2025. Enter the Void: Exploring with High Entropy Plans. arXiv preprint arXiv:2505.16787. (Under Submission)
- Gao, Y., Zhang, Q., Ding, D.W. and Zhao, D., 2024. Dream to Drive With Predictive Individual World Model. IEEE Transactions on Intelligent Vehicles.

Discussions

Paper 1: Bar, A., Zhou, G., Tran, D., Darrell, T. and LeCun, Y., 2025. Navigation world models. In Proceedings of the Computer Vision and Pattern Recognition Conference (pp. 15791-15801).

Paper 2: Gao, S., Zhou, S., Du, Y., Zhang, J. and Gan, C., 2025. Adaworld: Learning adaptable world models with latent actions. arXiv preprint arXiv:2503.18938.